Android - quizz

TURPIN Killian – BTS SIO

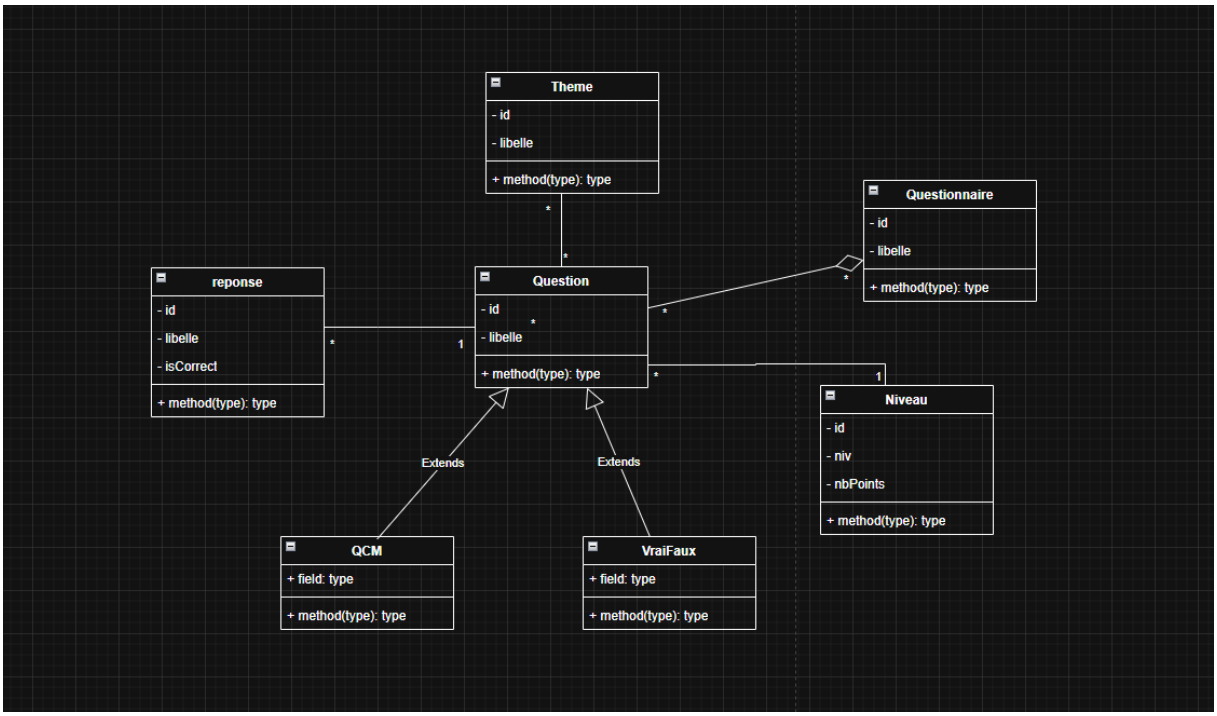
Table des matières

Schéma UML 2
Création de l'application web Cake PHP 2
Fichiers de migrations 3
Contrôleurs 5
Fonctions JSON 7
Les templates 9
Eléments 10
Création de l'application Android 16
Contrôle de la navigation 16
Ecran 1 : Accueil 18
Ecran 2 : Choix du Quiz 20
Modèle : Charger la liste des questionnaires 21
La vue de la page 22
Ecran 3 : Jeu en cours 25
Le modèle 25
La vue : 28
API JSON – Cake PHP & Android 33

Schéma UML

Le projet est un système de Quiz qui se compose de questions avec 4 possibilités de réponses avec des niveaux et des thèmes différents.

Le questionnaire se compose de plusieurs questions qui peut être du type QCM ou un Vrai ou Faux, dans ce cas, 2 possibilités de réponses sont possibles.



Création de l'application web Cake PHP

L'application Cake PHP qui aidera de base pour faire le quiz est fait sous **PostgreSQL**

Les fichiers de migrations ainsi que les modèles, contrôleur, et la vue ont été générés automatiquement via cmd avec la commande **cake bake** pour un gain de temps sur le reste du projet.

```
276     */
277     'Datasources' => [
278         /*
279          * These configurations should contain permanent settings used
280          * by all environments.
281          *
282          * The values in app_local.php will override any values set here
283          * and should be used for local and per-environment configurations.
284          *
285          * Environment variable-based configurations can be loaded here or
286          * in app_local.php depending on the application's needs.
287          */
288         'default' => [
289             'className' => Connection::class,
290             'driver' => Postgres::class,
291             'persistent' => false,
292             'timezone' => 'UTC',
293
294             /*
295              * For MariaDB/MySQL the internal default changed from utf8 to utf8mb4, aka full utf-8 support
296              */
297             'encoding' => 'utf8',
298
299             /*
300              * If your MySQL server is configured with `skip-character-set-client-handshake`
301              * then you MUST use the `flags` config to set your charset encoding.
302              * For e.g. ``'flags' => [\PDO::MYSQL_ATTR_INIT_COMMAND => 'SET NAMES utf8mb4']`
303              */
304             'flags' => [],
305             'cacheMetadata' => true,
306             'log' => false,
307
308             /*
```

Le fichier app.php avec la modification de « encoding » et « driver » pour correspondra à Postgres et à utf8

La base de données et l'utilisateur a été créé avec SQL Shell (Pgsql) – Ne pas oublier de connecter la base de données dans app_local.php avec les informations correspondantes à l'utilisateur créé pour la base de données.

```
40      'Datasources' => [
41          'default' => [
42              'host' => 'localhost',
43              /*
44               * CakePHP will use the default DB port based on the driver selected
45               * MySQL on MAMP uses port 8889, MAMP users will want to uncomment
46               * the following line and set the port accordingly
47               */
48              //'port' => 'non_standard_port_number',
49
50              'username' => 'admquizz',
51              'password' => 'AdmQuizz974/',
52
53              'database' => 'appquizz',
54              /*
55               * If not using the default 'public' schema with the PostgreSQL driver
56               * set it here.
57               */
58              //'schema' => 'myapp',
```

Fichiers de migrations

Quelques exemples de fichiers de migrations fait pour l'application :

CreateQuestions

```
 */
public function change(): void
{
    $table = $this->table('questions');
    $table->addColumn('libelle', 'string', [
        'default' => null,
        'limit' => 255,
        'null' => false,
    ]);

    $table->addColumn('type_id', 'integer', [
        'null' => false
    ]);

    $table->addForeignKey('type_id', 'types', 'id');

    $table->addColumn('niveau_id', 'integer', [
        'null' => false
    ]);

    $table->addForeignKey('niveau_id', 'niveaux', 'id');

    $table->addColumn('created', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addColumn('modified', 'datetime', [
        'default' => null,
        'null' => false,
    ]);


    $table->create();
```

CreateReponses :

```php
public function change(): void
{
    $table = $this->table('reponses');
    $table->addColumn('libelle', 'string', [
        'default' => null,
        'limit' => 255,
        'null' => false,
    ]);
    $table->addColumn('iscorrect', 'boolean', [
        'default' => null,
        'null' => false,
    ]);

    $table->addColumn('question_id', 'integer', [
        'null' => false
    ]);

    $table->addForeignKey('question_id', 'questions', 'id');

    $table->addColumn('created', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addColumn('modified', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->create();
}
```

CreateQuestionsThemes :

```php
 */
public function change(): void
{
    $table = $this->table('questions_themes');
    $table->addColumn('question_id', 'integer', [
        'default' => null,
        'limit' => 11,
        'null' => false,
    ]);
    $table->addForeignKey('question_id', 'questions', 'id');

    $table->addColumn('theme_id', 'integer', [
        'default' => null,
        'limit' => 11,
        'null' => false,
    ]);

    $table->addForeignKey('theme_id', 'themes', 'id');

    $table->addColumn('created', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addColumn('modified', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->create();
}
```

CreateQuestionnairesQuestions :

```
public function change(): void
{
    $table = $this->table('questionnaires_questions');
    $table->addColumn('questionnaire_id', 'integer', [
        'default' => null,
        'limit' => 11,
        'null' => false,
    ]);

    $table->addForeignKey('questionnaire_id', 'questionnaires', 'id');


    $table->addColumn('question_id', 'integer', [
        'default' => null,
        'limit' => 11,
        'null' => false,
    ]);

    $table->addForeignKey('question_id', 'questions', 'id');

    $table->addColumn('created', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addColumn('modified', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->create();
}
```

Contrôleurs

Comme les contrôleurs ont été généré automatiquement, les méthodes ont été pour la plupart été fait automatiquement.

Voici une partie des méthodes des contrôleurs modifiés à la suite de la génération pour les besoins du projet.

QuestionnairesController :

```php
 */
public function index()
{
    $query = $this->Questionnaires->find()->contain([
                'Questions' => function ($q) {
                    return $q
                        ->select(['id']);
                }]);
    $questionnaires = $this->paginate($query);
    $test = $query->all();
    $this->set(compact('questionnaires', 'test'));
}

/**
 * View method
 *
 * @param string|null $id Questionnaire id.
 * @return \Cake\Http\Response|null|void Renders view
 * @throws \Cake\Datasource\Exception\RecordNotFoundException When record not found.
 */
public function view($id = null)
{
    $questionnaire = $this->Questionnaires->get($id, contain: ['Questions']);
    $this->set(compact('questionnaire'));
}
```

```php
 */
public function add()
{
    $questionnaire = $this->Questionnaires->newEmptyEntity();
    if ($this->request->is('post')) {
        $questionnaire = $this->Questionnaires->patchEntity($questionnaire, $this->request->getData());
        if ($this->Questionnaires->save($questionnaire)) {
            $this->Flash->success(__('The questionnaire has been saved.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The questionnaire could not be saved. Please, try again.'));
    }
    $questions = $this->Questionnaires->Questions->find('list', limit: 200)->all();
    $this->set(compact('questionnaire', 'questions'));
}

/**
 * Edit method
 *
 * @param string|null $id Questionnaire id.
 * @return \Cake\Http\Response|null|void Redirects on successful edit, renders view otherwise.
 * @throws \Cake\Datasource\Exception\RecordNotFoundException When record not found.
 */
public function edit($id = null)
{
    $questionnaire = $this->Questionnaires->get($id, contain: ['Questions']);
    if ($this->request->is(['patch', 'post', 'put'])) {
        $questionnaire = $this->Questionnaires->patchEntity($questionnaire, $this->request->getData());
        if ($this->Questionnaires->save($questionnaire)) {
            $this->Flash->success(__('The questionnaire has been saved.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The questionnaire could not be saved. Please, try again.'));
    }
    $questions = $this->Questionnaires->Questions->find('list', limit: 200)->all();
    $this->set(compact('questionnaire', 'questions'));
}
```

Nous avons voulu rajouter les données de la classe Questions dans Questionnaires avec les « contain » dans les méthodes index, add, edit, view

```php
public function indexJson() {
    $questionnaires = $this->Questionnaires->find('all')
            -> select([
                'Questionnaires.id',
                'Questionnaires.libelle',
                'date_creation'=> "TO_CHAR(Questionnaires.created,'DD/MM/YYYY')",
                // On compte les entrées dans la table de jointure
                'nb_questions' => 'COUNT(Questions.id)'
            ])
            ->leftJoinWith('Questions') // Cake fait le pont via questionnaire
            ->groupBy(['Questionnaires.id', 'Questionnaires.libelle'])
            ->toArray();

    $this->setRequest($this->getRequest()->withHeader('Accept', 'application/json'));
    $this->set(compact('questionnaires'));
    $this->response = $this->response->withType('application/json');
    $this->viewBuilder()
            ->setClassName('Json')
            ->setOption('serialize', ['questionnaires']);
}
```

Fonctions JSON

```php
public function viewJson($id = null) {
    $id = (int) $id;
    // On récupère le questionnaire par son ID avec toute sa hiérarchie
    $questionnaire = $this->Questionnaires->get($id, [
        'fields' => ['id','libelle'],
        'contain' => [
            'Questions' => [
                'fields' => ['id', 'libelle', 'type_id', 'niveau_id'],
                'niveaux' => [
                    'fields' => ['id', 'niv', 'nbpoints'],
                ],
                'Themes' => [
                    'fields' => ['id','libelle'],
                ],
                'Reponses' => [
                    'fields' => ['id','libelle','iscorrect','question_id'],
                ],
                'Types' => [
                    'fields' => ['id','libelle'],
                ]
            ]
        ]
    ]);
    $this->setRequest($this->getRequest()->withHeader('Accept', 'application/json'));
    $this->set(compact('questionnaire'));
    $this->response = $this->response->withType('application/json');
    $this->viewBuilder()
            ->setClassName('Json')
            ->setOption('serialize', ['questionnaire']);
}
```

2 fonctions qui serviront plus tard dans la récupération de données pour notre application Android. Elle sert principalement à transformer les données récupérées par la variable $questionnaire au format JSON.

QuestionsController :

```
 */
public function index()
{
    $query = $this->Questions->find()
        ->contain(['Types', 'niveaux']);
    $questions = $this->paginate($query);

    $this->set(compact('questions'));
}

/**
 * View method
 *
 * @param string|null $id Question id.
 * @return \Cake\Http\Response|null|void Renders view
 * @throws \Cake\Datasource\Exception\RecordNotFoundException When record not found.
 */
public function view($id = null)
{
    $question = $this->Questions->get($id, contain: ['Types', 'niveaux', 'Questionnaires', 'Themes', 'Reponses']);
    $this->set(compact('question'));
}
```

```
 */
public function add()
{
    $question = $this->Questions->newEmptyEntity();
    if ($this->request->is('post')) {
        $question = $this->Questions->patchEntity($question, $this->request->getData(),
                ['associated' => ['Reponses', 'Themes', 'Questionnaires']]);
        if ($this->Questions->save($question)) {
            $this->Flash->success(__('The question has been saved.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The question could not be saved. Please, try again.'));
    }
    $types = $this->Questions->Types->find('list', limit: 200)->all();
    $niveaux = $this->Questions->niveaux->find('list', limit: 200)->all();
    $questionnaires = $this->Questions->Questionnaires->find('list', limit: 200)->all();
    $themes = $this->Questions->Themes->find('list', limit: 200)->all();
    $this->set(compact('question', 'types', 'niveaux', 'questionnaires', 'themes'));
}
```

```php
public function edit($id = null) {
    $question = $this->Questions->get($id,
            contain: ['Questionnaires',
                'Themes',
                'Reponses' => function ($q) {
                        return $q
                                ->orderBy(['id asc']);
                    },
                    'Types']
            );
    if ($this->request->is(['patch', 'post', 'put'])) {
        $question = $this->Questions->patchEntity($question, $this->request->getData(),
                ['associated' => ['Reponses',
                    'Themes',
                    'Questionnaires']]);
        if ($this->Questions->save($question)) {
            //debug($question->getErrors());
            //debug($this->request->getData());die;
            $this->Flash->success(__('The question has been saved.'));

            return $this->redirect(['action' => 'edit', $id]);
        }
        $this->Flash->error(__('The question could not be saved. Please, try again.'));
    }
    $types = $this->Questions->Types->find('list', limit: 200)->all();
    $niveaux = $this->Questions->niveaux->find('list', limit: 200)->all();
    $questionnaires = $this->Questions->Questionnaires->find('list', limit: 200)->all();
    $themes = $this->Questions->Themes->find('list', limit: 200)->all();
    $this->set(compact('question', 'types', 'niveaux', 'questionnaires', 'themes'));
}
```

Les templates

Certains templates ont été modifiés pour pouvoir gérer l'interface du quiz sur le site fait par Cake. C'est notamment le cas pour la partie Questions dont le script jquery371min a été rajouté (voir la doc. Blog App) ainsi qu'un script qui gère le choix des types de questions *(QCM/VraiFaux)*

```html
<script>
    document.addEventListener('DOMContentLoaded', function () {
        console.log("JS Chargé !"); // Si tu ne vois pas ça, le script n'est pas lu

        const selectors = document.querySelectorAll('.type-selector');
        const divQcm = document.getElementById('section-qcm');
        const divVf = document.getElementById('section-vrai-faux');

        console.log("Nombre de boutons trouvés :", selectors.length);

        selectors.forEach(radio => {
            radio.addEventListener('change', function () {
                console.log("Changement détecté sur :", this.id); // Vérifie si le clic est capté

                // Masquer tout
                if (divQcm) {
                    divQcm.style.display = 'none';
                    divQcm.querySelectorAll('input, select, textarea').forEach(input => {
                        input.disabled = true;
                    });
                }
                if (divVf) {
                    divVf.style.display = 'none';
                    divVf.querySelectorAll('input, select, textarea').forEach(input => {
                        input.disabled = true;
                    });
                }
                // Afficher selon la value
                if (this.id.includes('QCM')) {
                    console.log("Affichage QCM validé");
                    divQcm.style.display = 'block';
                    divQcm.querySelectorAll('input, select, textarea').forEach(input => {
                        input.disabled = false;
                    });
                } else if (this.id.includes('Vrai/Faux')) {
                    console.log("Affichage Vrai/Faux validé");
                    divVf.style.display = 'block';
                    divVf.querySelectorAll('input, select, textarea').forEach(input => {
                        input.disabled = false;
                    });
                }
            });
        });
```

```
<?= $this->Form->create($question) ?>
<fieldset>
    <legend><?= __('Add Question') ?></legend>
    <?= $this->Form->control('libelle'); ?>
    <?= $this->Form->control('niveau_id', ['options' => $niveaux]); ?>
    <?=
    $this->Form->control('questionnaires._ids',
            ['options' => $questionnaires,
                'multiple' => 'checkbox']);
    ?>
    <?=
    $this->Form->control('themes._ids',
            ['options' => $themes,
                'multiple' => 'checkbox']);
    ?>
    <div style="margin: 20px 0; padding: 15px; border: 1px solid #ddd; border-radius: 8px;">
        <label style="font-weight: bold; display: block; margin-bottom: 15px;">Type de question :</la

        <div style="display: flex; gap: 25px; align-items: center;">
            <?php
            foreach ($types as $id => $label):
                // On remplace les espaces et slashs par des tirets, et on met en minuscule
                $idUnique = 'type-' . $label;
                ?>
                <label style="display: flex; align-items: center; cursor: pointer; gap: 8px; margin:
                    <?=
                    $this->Form->radio('type_id',
                            [['value' => $id, 'text' => $label]],
                            [
                                'hiddenField' => false,
                                'class' => 'type-selector',
                                'id' => $idUnique, // Deviendra "type-qcm" ou "type-vrai-faux"
                                'label' => false, // On gère le label manuellement pour le centrage
                                'style' => 'margin: 0; width: 18px; height: 18px; cursor: pointer;'
                            ]
                    );
                    ?>
                    <span style="font-size: 15px; line-height: 1;"><?= h($label) ?></span>
                </label>
            <?php endforeach; ?>
        </div>
    </div>
    <div id="section-qcm" style="display: none; border: 1px solid #333; padding: 15px;">
```

Eléments

Pour la gestion du changement entre QCM et Vrai/Faux cela se passe dans la partie Elements :

Qcm.php :

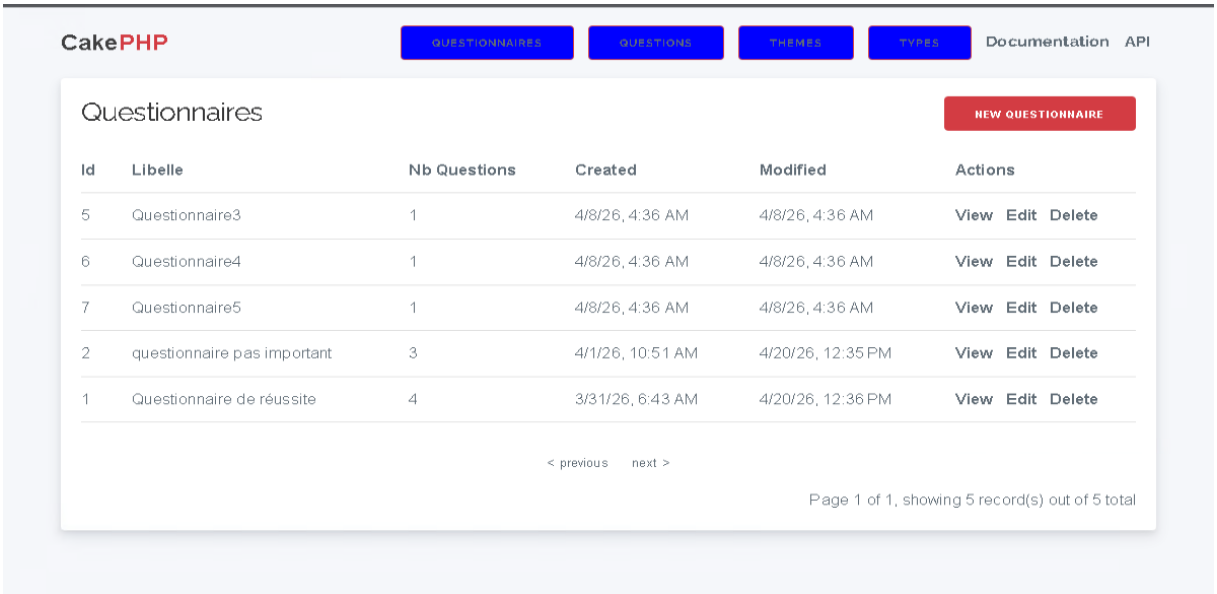
```
<h1>Réponse QCM </h1>
    <div style="margin: 20px 0;">
    <label style="display: block; margin-bottom: 15px; font-weight: bold;">Associer des réponses :</label>

    <?php $rep = [0,1,2,3];
    foreach ($rep as $value): ?>
        <div style="border: 1px solid #e6eef6; padding: 14px; margin-bottom: 12px; background:#ffffff ; border-radius: 8px; box-shadow: 0 1p
            <div style="display: flex; justify-content: space-between; align-items: center; gap: 12px; min-height: 48px;">
                <div style="display: flex; align-items: center; flex: 1; gap: 10px; font-size: 15px; color: #111827;">
                    <span style="font-weight: 600; margin-right: 6px;">Reponse <?= $value+1 ?> :</span>

                    <?=
                    $this->Form->control("reponses.$value.libelle", [
                        'value' => $value,
                        'style' => 'margin: 0; margin-right: 8px; width: 100%; height: 36px; padding: 6px 8px; border: 1px solid #cbd5e1; bor
                        'id' => "reponse-{$value}",
                        'hiddenField' => false
                    ])
                    ?>
                    <?=
                     $this->Form->radio("reponses.$value.iscorrect",
                            [
                                ['value' => 1, 'text' => 'Bonne reponse'],
                            ],
                            [
                                'id' => "icor",
                                'style' => 'margin: 0; padding: 6px 10px; height: auto; border: 1px solid #cbd5e1; border-radius: 6px; backgr
                            ])
                    ?>
                </div>


            </div>
        </div>
<?php endforeach; ?>
</div>
```

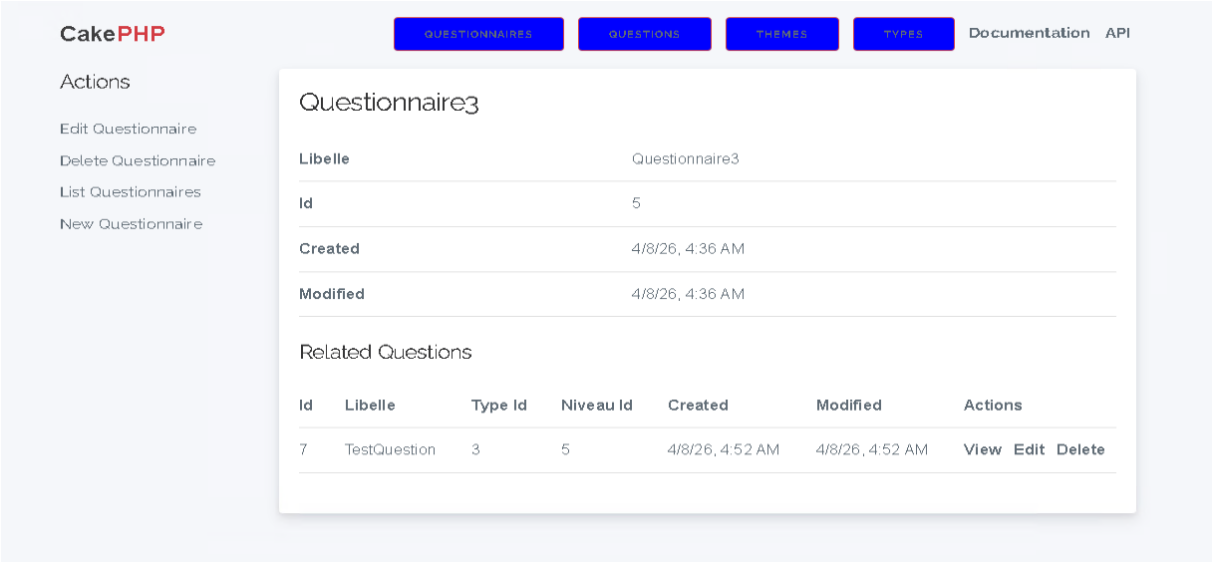
Vraifaux.php :

```php
<?php
$cpt = 0;
$rep = [0,1];
foreach ($rep as $value):
    ?>
    <div style="border: 1px solid #ddd; padding: 10px 15px; margin-bottom: 10px; background: #f9f9f9; border-radius: 4px;">
        <div style="display: flex; align-items: center; gap: 15px; min-height: 45px;">

            <span style="font-weight: bold; white-space: nowrap; min-width: 90px;">Réponse <?= $cpt + 1 ?> :</span>

            <div style="flex: 1; display: flex; align-items: center;">
                <?php
                if ($cpt == 0) {
                    $place = "Faux";
                } else {
                    $place = "Vrai";
                }
                ?>
                <?=
                $this->Form->control("reponses.$value.libelle", [
                    'label' => false,
                    'templates' => [
                        'inputContainer' => '{{content}}' // Supprime la div qui décale tout
                    ],
                    'value' => $place,
                    'style' => 'width: 100%; padding: 8px 12px; border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; margin: 0; line-he
                ])
                ?>
            </div>

            <div style="display: flex; align-items: center; background: #fff; border: 1px solid #ddd; padding: 7px 12px; border-radius: 4px; white-sp
                <?=
                $this->Form->hidden("reponses.$value.iscorrect", [
                    'value' => 0,
                    'secure' => false]);
                ?>
                <?=
                $this->Form->radio("reponses.$value.iscorrect",
                        [
                            ['value' => 1, 'text' => 'Bonne réponse']
                        ],
                        [
                            'label' => [
```

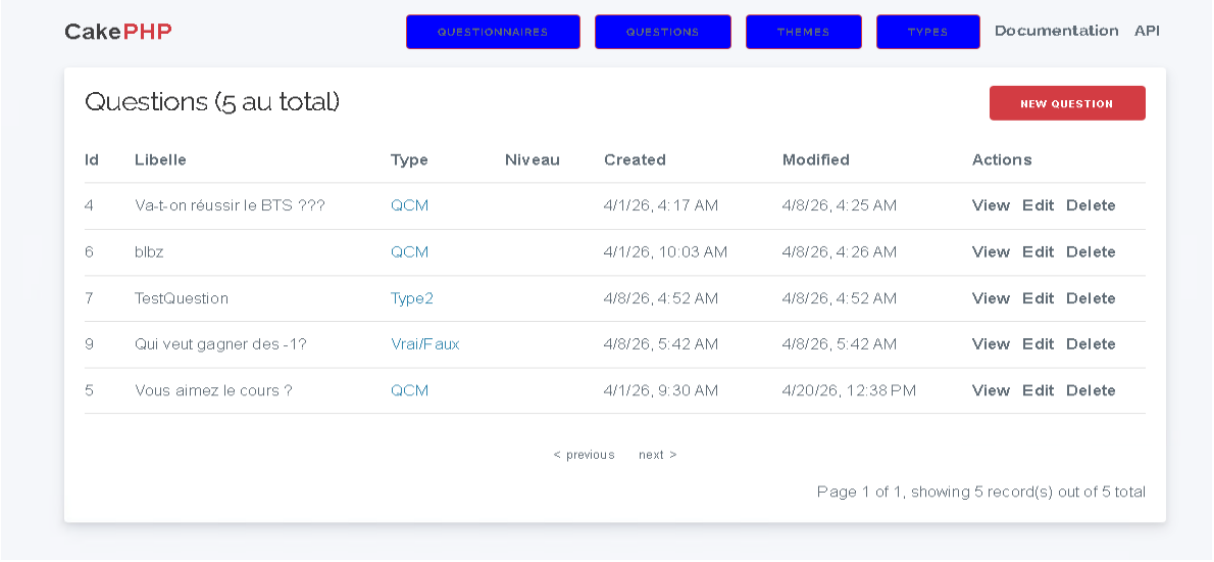
Résultat sur certaines pages :

Questionnaires :




CakePHP
QUESTIONNAIRES
QUESTIONS
THEMES
TYPES
Documentation
API
Actions
Edit Questionnaire
Delete Questionnaire
List Questionnaires
New Questionnaire
Questionnaire3
Libelle
Questionnaire3
Id
5
Created
4/8/26, 4:36 AM
Modified
4/8/26, 4:36 AM
Related Questions
Id
Libelle
Type Id
Niveau Id
Created
Modified
Actions
7
TestQuestion
3
5
4/8/26, 4:52 AM
4/8/26, 4:52 AM
View Edit Delete

Questions :


CakePHP
QUESTIONNAIRES
QUESTIONS
THEMES
TYPES
Documentation
API
Questions (5 au total)
NEW QUESTION
Id
Libelle
Type
Niveau
Created
Modified
Actions
4
Va-t-on réussir le BTS ???
QCM
4/1/26, 4:17 AM
4/8/26, 4:25 AM
View Edit Delete
6
blbz
QCM
4/1/26, 10:03 AM
4/8/26, 4:26 AM
View Edit Delete
7
TestQuestion
Type2
4/8/26, 4:52 AM
4/8/26, 4:52 AM
View Edit Delete
9
Qui veut gagner des -1?
Vrai/Faux
4/8/26, 5:42 AM
4/8/26, 5:42 AM
View Edit Delete
5
Vous aimez le cours ?
QCM
4/1/26, 9:30 AM
4/20/26, 12:38 PM
View Edit Delete
< previous
next >
Page 1 of 1, showing 5 record(s) out of 5 total

Actions

Edit Question
Delete Question
List Questions
New Question

Va-t-on réussir le BTS ???

Libelle	Va-t-on réussir le BTS ???
Type	QCM
Niveau	
Id	4
Created	4/1/26, 4:17 AM
Modified	4/8/26, 4:25 AM

Related Questionnaires

Id	Libelle	Created	Modified	Actions
1	Questionnaire de réussite	3/31/26, 6:43 AM	4/20/26, 12:36 PM	View Edit Delete

Related Themes

Related Reponses

Id	Libelle	IsCorrect	Created	Modified	Actions
5	Un jour maybe	1	4/1/26, 4:19 AM	4/1/26, 4:19 AM	View Edit Delete
6	Nooooooon		4/1/26, 4:20 AM	4/1/26, 4:20 AM	View Edit Delete

Activer Windo
Accédez aux paran

Delete
List Questions

Modifier la question

Libelle

Va-t-on réussir le BTS ???

Type de question :

QCM

Niveau

Difficile

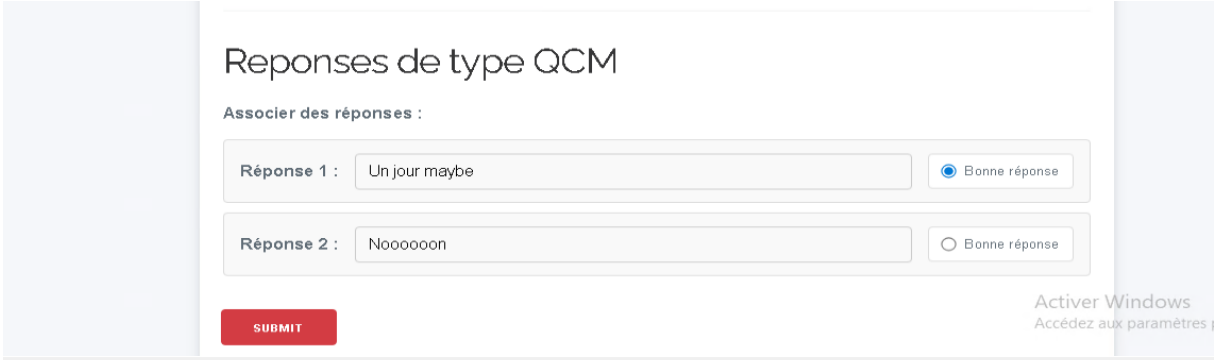
Questionnaires

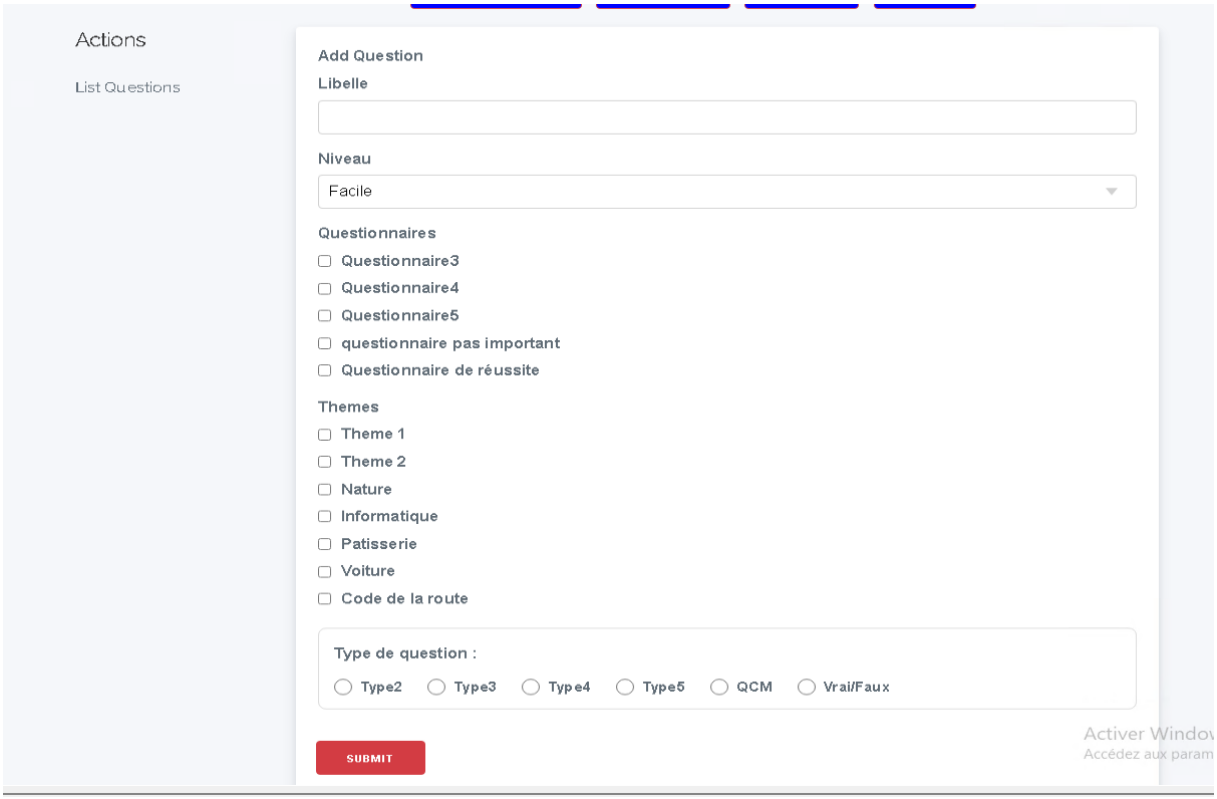
- [ ] Questionnaire3
- [ ] Questionnaire4
- [ ] Questionnaire5
- [ ] questionnaire pas important
- [x] Questionnaire de réussite

Themes

- [ ] Theme 1
- [ ] Theme 2
- [ ] Nature
- [ ] Informatique
- [ ] Patisserie
- [ ] Voiture
- [ ] Code de la route

Activer Windows
Accédez aux paramètres pour act


Reponses de type QCM
Associer des réponses :
Réponse 1 :
Un jour maybe
Bonne réponse
Réponse 2 :
Noooooon
Bonne réponse
SUBMIT
Activer Windows


Actions
List Questions
Add Question
Libelle
Niveau
Facile
Questionnaires
Questionnaire3
Questionnaire4
Questionnaire5
questionnaire pas important
Questionnaire de réussite
Themes
Theme 1
Theme 2
Nature
Informatique
Patisserie
Voiture
Code de la route
Type de question :
Type2
Type3
Type4
Type5
QCM
Vrai/Faux
SUBMIT

Thèmes :

CakePHP

QUESTIONNAIRES QUESTIONS THEMES TYPES Documentation API

Themes

NEW THEME

Id	Libelle	Created	Modified	Actions
1	Theme 1	3/31/26, 6:28 AM	3/31/26, 6:28 AM	View Edit Delete
2	Theme 2	4/1/26, 4:12 AM	4/1/26, 4:12 AM	View Edit Delete
3	Nature	4/8/26, 4:36 AM	4/8/26, 4:36 AM	View Edit Delete
4	Informatique	4/8/26, 4:36 AM	4/8/26, 4:36 AM	View Edit Delete
5	Patisserie	4/8/26, 4:36 AM	4/8/26, 4:36 AM	View Edit Delete
6	Voiture	4/8/26, 4:36 AM	4/8/26, 4:36 AM	View Edit Delete
7	Code de la route	4/8/26, 4:36 AM	4/8/26, 4:36 AM	View Edit Delete

< previous next >

Page 1 of 1, showing 7 record(s) out of 7 total

CakePHP

QUESTIONNAIRES QUESTIONS THEMES TYPES Documentation API

Actions

- Edit Theme
- Delete Theme
- List Themes
- New Theme

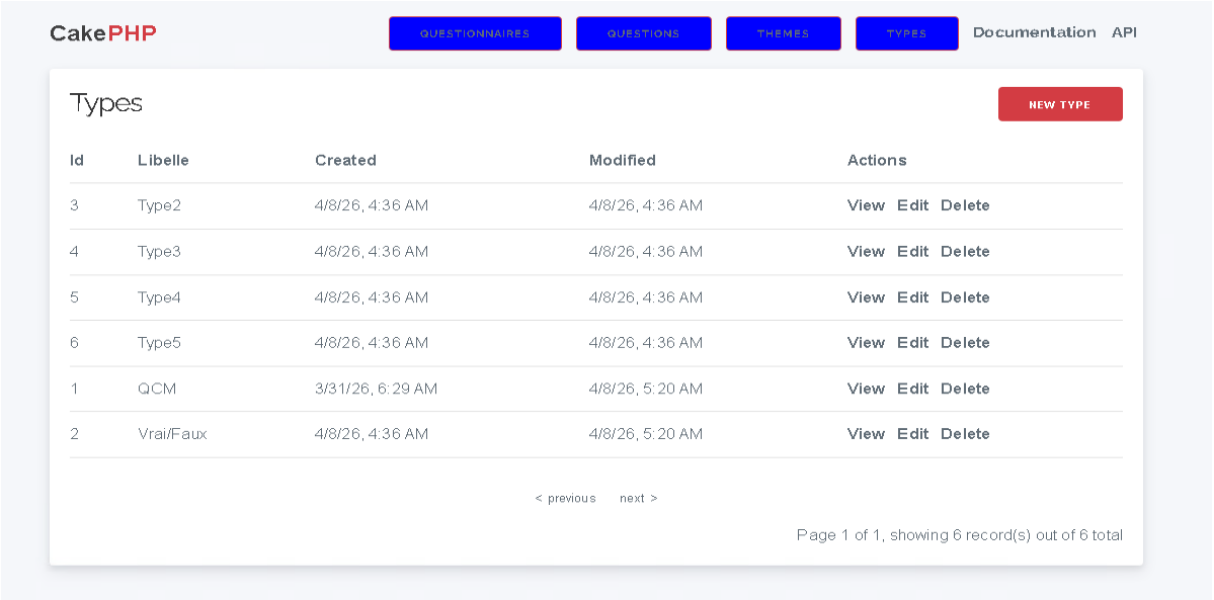
Theme 1

Libelle	Theme 1
Id	1
Created	3/31/26, 6:28 AM
Modified	3/31/26, 6:28 AM

Related Questions

Id	Libelle	Type Id	Niveau Id	Created	Modified	Actions
7	TestQuestion	3	5	4/8/26, 4:52 AM	4/8/26, 4:52 AM	View Edit Delete

Type :



Il est important ici d'avoir un type « QCM » et « Vrai/Faux » avec les caractères bien spécifiques afin d'avoir dans le choix des questions la bonne section qui fera apparaître le choix des réponses dans le format souhaité (QCM / VraiFaux)

Une fois que l'application web est achevé, passons sous Android Studio.

Création de l'application Android

Après avoir installé Android Studio, puis suivi le tutoriel d'exemple, voici toutes les pages et l'explication du code.

Contrôle de la navigation

```
1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            QuizzTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    QuizApp()
                }
            }
        }
    }
}

1 Usage
@Preview(showBackground = true)
@Composable
fun QuizApp() {
    val navController = rememberNavController()

    // Gestionnaire de navigation (Le GPS de l'app)
    NavHost(navController = navController, startDestination = "accueil") {
        composable( route = "accueil") { EcranAccueil(navController) }
        composable( route = "choix") { EcranChoix(navController) }

        composable( route = "page_jeu/{id}") { backStackEntry ->
            val quizzId = backStackEntry.arguments?.getString( key = "id")?.toIntOrNull() ?: 0
            // Pour l'instant, on redirige vers l'écran de jeu
            EcranJeu( quizId = quizzId, navController)
        }
    }
}
```

Le gestionnaire de la navigation sur l’application

Ecran 1 : Accueil

Dans QuizzViews.kt nous avons la vue de toutes les pages de l'application :

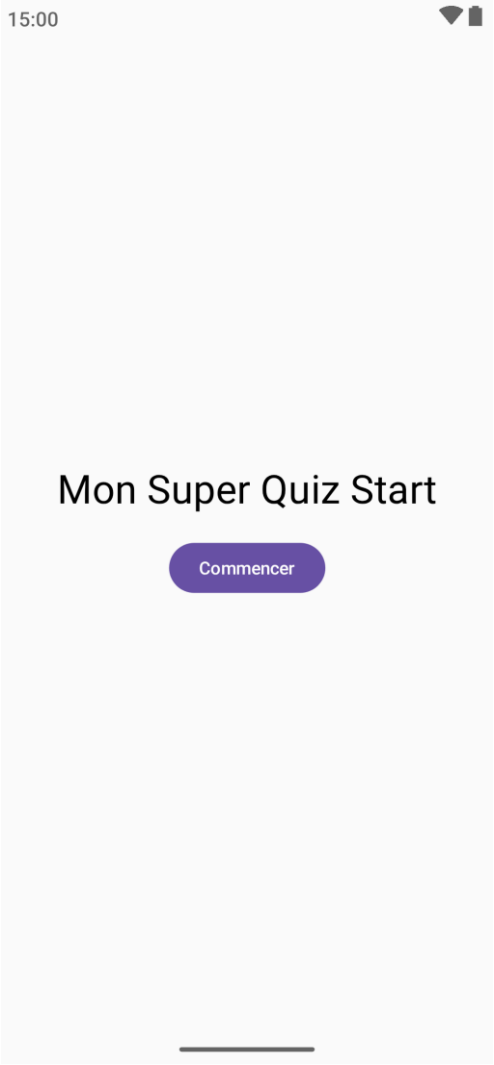
```
// ==================================================================
// ÉCRAN 1 : ACCUEIL
// ==================================================================

// COMPOSANT PARENT : Gère la navigation
1 Usage
@Composable
fun EcranAccueil(navController: NavHostController) {
    // On appelle une version "UI seule" pour faciliter la preview
    ContenuAccueil(onCommencerClick = { navController.navigate( route = "choix") })
}

// COMPOSANT ENFANT : Gère uniquement le visuel (Dumb Component)
2 Usages
@Composable
fun ContenuAccueil(onCommencerClick: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Mon Super Quiz Start", fontSize = 32.sp)
        Spacer(modifier = Modifier.height(20.dp))
        Button(onClick = onCommencerClick) {
            Text("Commencer")
        }
    }
}
```

On appelle Column lorsqu'il faut pouvoir imbriquer des éléments en colonnes notamment le verticalArrangement et horizontalAlignment pour centrer verticalement et horizontalement les éléments du bloc.

Dans ce bloc, nous avons un Text, Spacer (Espace) et un bouton qui a une action onCommencerClick au clic du bouton : cette action est écrite un peu plutôt dans ContenuAccueil et permet de rediriger l'utilisateur vers une autre page.

```
// ======================================================================
// PREVIEWS (Pour tester le design sans lancer l'app)
// ======================================================================

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun PreviewAccueil() {
    // On peut tester le visuel sans avoir besoin de Navigation !
    ContenuAccueil(onCommencerClick = {})
}
```

La partie preview pour voir le rendu de l’application sans passer par le démarrage de l’émulateur.



La preview de l’accueil

Ecran 2 : Choix du Quiz

```
// ====================================================================
// ÉCRAN 2 : CHOIX DU QUIZ
// ====================================================================
@Composable
fun EcranChoix(navController: NavHostController, viewModel: QuizzViewModel = viewModel()) {
    // On déclenche le chargement des données au démarrage de l'écran
    LaunchedEffect( key1 = Unit) {
        viewModel.chargerListeQuestionnaires()
    }

    // On passe les états du ViewModel à la Vue "passive"
    ContenuChoix(
        listQuizz = viewModel.listeQuizApercu.value,
        isLoading = viewModel.isLoading.value,
        onSelected = { id ->
            navController.navigate( route = "page_jeu/$id")
        }
    )
}
```

Une fois qu'on a appuyé sur le bouton « Commencer » de la précédente page, nous avons deux fonctions qui servent à la gestion d'un choix de Quiz.

D'abord on charge le modèle qui permet de récupérer la liste des questionnaires disponibles et on passe en état dans ContenuChoix avec listQuizz qui récupère la liste des différents questionnaires.

```
class QuizzViewModel(application: Application) : AndroidViewModel(application) {

    // ================= ÉTATS (Observables par l'interface) =================

    // La liste des quiz pour l'écran de choix
    2 Usages
    var listeQuizApercu = mutableStateOf<List<QuizzApercu>>( value = emptyList())
```

Modèle : Charger la liste des questionnaires

```
// ================= ACTIONS (Appelées par l'interface) =================

// Charge la liste de tous les quiz disponibles
1 Usage
fun chargerListeQuestionnaires() {
    isLoading.value = true
    viewModelScope.launch { // Coroutine : s'exécute en arrière-plan
        try {
            val reponse = RetrofitClient.instance.getListeQuiz()
            listeQuizApercu.value = reponse.listeQuestionnaires
        } catch (e: HttpException) {
            // Si c'est une erreur HTTP (comme 404, 500)
            val urlAppelee = e.response()?.raw()?.request?.url
            val messageErreur = e.response()?.errorBody()?.string()

            Log.e( tag = "API_ERREUR",  msg = "🔴 L'URL qui a planté : $urlAppelee")
            Log.e( tag = "API_ERREUR",  msg = "🔴 Code erreur : ${e.code()}")
            //Log.e("API_ERREUR", "🔴 Réponse du serveur : $messageErreur")
        }catch (e: Exception) {
            e.printStackTrace()
        } finally {
            isLoading.value = false
        }
    }
}
```

Dans le modèle, on vérifie si les informations récupérées via le lien de l'API fonctionnent correction (plus d'information dans la partie API JSON)

La vue de la page

```
// COMPOSANT ENFANT : Affiche la liste des questionnaires
@Composable
fun ContenuChoix(
    listQuizz: List<QuizzApercu>,
    isLoading: Boolean,
    onSelected: (Int) -> Unit
) {
    Column(
        modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text("Choisissez votre Quiz", fontSize = 30.sp)
        Spacer(modifier = Modifier.height(20.dp))

        if (isLoading) {
            CircularProgressIndicator()
        } else {
            LazyColumn(
                modifier = Modifier.fillMaxWidth(),
                horizontalAlignment = Alignment.CenterHorizontally // Centre les colonnes dans la liste
            ) {
                items(listQuizz) { quiz ->
                    Column(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalAlignment = Alignment.CenterHorizontally // Centre le bouton et le texte entre eux
                    ) {
                        Button(
                            onClick = { onSelected(quiz.id) },
                            modifier = Modifier
                                .fillMaxWidth( fraction = 0.8f)
                                .padding(top = 8.dp, bottom = 4.dp)
                        ) {
                            Text(quiz.libelle + " : " + quiz.nbQuestions + " questions")
                        }

                        Text(
                            text = "Créé le " + quiz.datecreation,
                            fontSize = 10.sp,
                            textAlign = TextAlign.Center
                        )
                        Spacer(modifier = Modifier.height(20.dp))
```

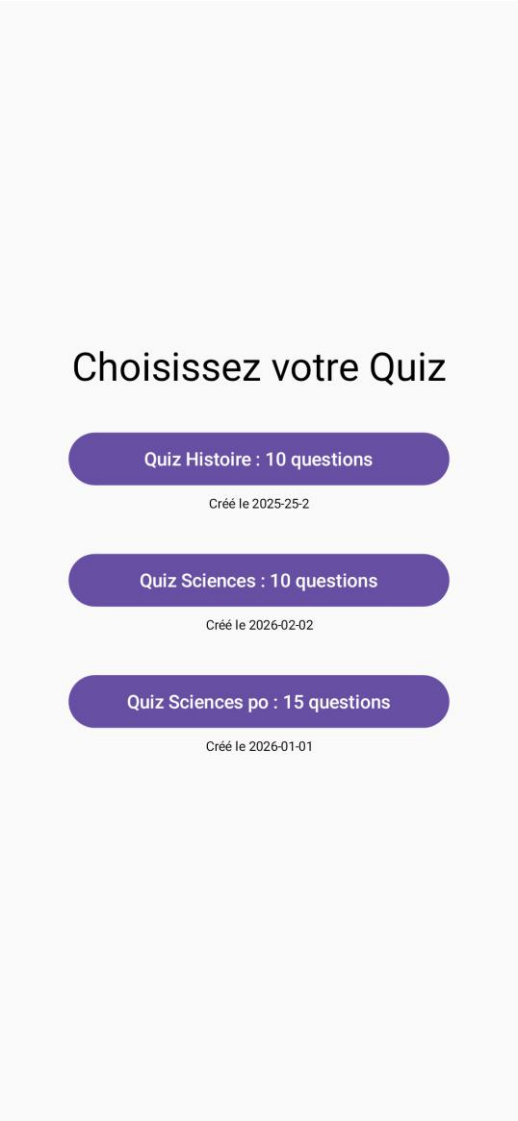
Pour la partie visuelle, le isLoading, permet d'afficher un indicateur circulaire d'android et reste à true tant que les données sont en cours de chargement. Autrement il affiche les colonnes, les items, boutons et textes correspondants.

```
// --- LA PREVIEW CHOIX ---
@Preview(showBackground = true)
@Composable
fun PreviewChoix() {
    val fakeQuizzes = listOf(
        QuizzApercu(id = 1, libelle = "Quiz Histoire", datecreation = "2025-25-2", nbQuestions = 10),
        QuizzApercu(id = 2, libelle = "Quiz Sciences", datecreation = "2026-02-02", nbQuestions = 10),
        QuizzApercu(id = 3, libelle = "Quiz Sciences po", datecreation = "2026-01-01", nbQuestions = 15)
    )

    ContenuChoix(
        listQuizz = fakeQuizzes,
        isLoading = false,
        onSelected = {}
    )
```

La partie preview pour avoir un aperçu de la page. On rajoute des « faux quiz » préremplies pour un aperçu global.

Voici le modèle de QuizzApercu de QuizzModel.kt :

```
// Version "légère" d'un Quiz (utilisée pour afficher la liste de choix)
8 Usages
data class QuizzApercu(
    val id: Int,
    val libelle: String,
    val datecreation: String,
    @SerializedName( value = "nb_questions") val nbQuestions: Int
)
```



Résultat de la preview de la page EcranChoix

Ecran 3 : Jeu en cours

Enfin, la partie la plus longue mais la plus importante du projet, c'est la gestion du jeu en cours avec notamment la question à afficher, les réponses, et le nombre de score et le score total à compter.

Le modèle

Les variables QuizzViewModel.kt :

```
// Le quiz actuellement joué
3 Usages
var quizActuel = mutableStateOf<QuizzDetail?>( value = null)

// Variables de progression du jeu
4 Usages
var indexQuestionActuelle = mutableStateOf( value = 0)
4 Usages
var score = mutableStateOf( value = 0)
3 Usages
var scoreTotal = mutableStateOf( value = 0)
3 Usages
var isFinished = mutableStateOf( value = false)
6 Usages
var isLoading = mutableStateOf( value = false)
```

Le chargement des détails du quiz durant la partie.

```
// Charge un quiz spécifique via son ID
1 Usage
fun chargerQuizDetail(id: Int) {
    // 1. Réinitialisation des stats de la partie
    indexQuestionActuelle.value = 0
    score.value = 0
    scoreTotal.value = 0
    isFinished.value = false
    isLoading.value = true

    // 2. Appel API
    viewModelScope.launch {
        try {
            val reponse = RetrofitClient.instance.getQuizDetail(id)
            quizActuel.value = reponse.detailQuestionnaire
            Log.i( tag = "QuizViewModel",  msg = "Quiz chargé : ${reponse.detailQuestionnaire.libelle}")
        } catch (e: HttpException) {
            // Si c'est une erreur HTTP (comme 404, 500)
            val urlAppelee = e.response()?.raw()?.request?.url
            val messageErreur = e.response()?.errorBody()?.string()

            Log.e( tag = "API_ERREUR",  msg = "🔴 L'URL qui a planté : $urlAppelee")
            Log.e( tag = "API_ERREUR",  msg = "🔴 Code erreur : ${e.code()}")
            //Log.e("API_ERREUR", "🔴 Réponse du serveur : $messageErreur")
        }catch (e: Exception) {
            e.printStackTrace()

        } finally {
            isLoading.value = false
        }
    }
}
```

Le passage à la prochaine question ET/OU la fin du quiz :

```
// Passe à la question suivante ou termine le quiz
1 Usage
fun passerALaQuestionSuivante() {
    val quiz = quizActuel.value
    if (quiz != null && indexQuestionActuelle.value < quiz.questions.size - 1) {
        indexQuestionActuelle.value++
    } else {
        isFinished.value = true // Déclenche l'écran de fin
    }
}
```

Les données complètes du modèle dans QuizzModel.kt

```
// Version "complète" d'un Quiz (utilisée pour jouer)
5 Usages
data class QuizzDetail(
    val id: Int,
    val libelle: String,
    val questions: List<Question>
)

5 Usages
data class Question(
    val id: Int,
    val libelle: String,
    val niveaux: Niveau, // Le barème de points
    val themes: List<Theme>,
    val reponses: List<Reponse>,
    val type: TypeQuestion // Permet de savoir si c'est un QCM ou Vrai/Faux
)

8 Usages
data class Reponse(
    val id: Int,
    val libelle: String,
    @SerializedName( value = "iscorrect") val estCorrecte: Boolean
)

4 Usages
data class Niveau(
    val id: Int,
    @SerializedName( value = "niv") val nom: String,
    val nbpoints: Int
)

4 Usages
data class Theme(val id: Int, val libelle: String)

4 Usages
data class TypeQuestion(
    val id: Int,
    val libelle: String
)
```

On ajoute les paramètres nécessaire au fonctionnement avec l'id du quiz et on demande à le faire charger.

```
// COMPOSANT PARENT : Logique métier et gestion du score
@Composable
fun EcranJeu(
    quizId: Int,
    navController: NavHostController,
    viewModel: QuizzViewModel = viewModel()
) {
    // 1. On demande au contrôleur de charger le quiz correspondant à l'ID
    LaunchedEffect( key1 = quizId) {
        viewModel.chargerQuizDetail(quizId)
    }
```

On récupère les états du QuizzViewModel

```
    // 2. On récupère les états du contrôleur
    val quiz = viewModel.quizActuel.value
    val isFinished = viewModel.isFinished.value

    // On récupère la question précise selon l'index
    val questionActuelle = if (isFinished) null else quiz?.questions?.getOrNull( index = viewModel.indexQuestionActuelle.value)
```

On fait appelle à la vue et on calcul le score en cas de bonne ou mauvaise réponse.

```
    // 3. ON APPELLE LE CONTENU (La Vue)
    ContenuJeu(
        quizLibelle = quiz?.libelle ?: "",
        question = questionActuelle,
        isLoading = viewModel.isLoading.value,
        score = viewModel.score.value,
        scoreTotal = viewModel.scoreTotal.value,
        onBackClick = { navController.popBackStack() },
        onReponseClick = { reponseId ->
            // Vérification de la réponse et calcul du score
            val reponseChoisie = questionActuelle?.reponses?.find { it.id == reponseId }
            Log.w( tag = "id reponse :", msg = reponseId.toString())
            Log.w( tag = "icorrect reponse :", msg = reponseChoisie?.estCorrecte.toString())
            Log.w( tag = "score reponse :", msg = questionActuelle?.niveaux?.nbpoints.toString())
            if(reponseChoisie?.estCorrecte == true){
                viewModel.score.value += questionActuelle.niveaux.nbpoints
            } else {
                viewModel.score.value += 0
            }
            viewModel.scoreTotal.value += questionActuelle!!.niveaux!!.nbpoints
            viewModel.passerALaQuestionSuivante()
        }
    )
```

La vue :

Toute la partie visuelle où on affiche le titre des questions, les boutons des réponses de type QCM ou Vrai/Faux ainsi que l'écran de fin avec le résultat final :

```
) {
    if (isLoading) {
        CircularProgressIndicator()
    } else if (question != null) {
        // 0. le theme

        // 1. Le titre de la question
        Text(
            text = question.libelle,
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            textAlign = TextAlign.Center
        )

        Spacer(modifier = Modifier.height(32.dp))
```

```
// 2. LA LOGIQUE DES BOUTONS (C'est ici qu'on décide 2 ou 4)
if (question.type.libelle.contains( other = "Vrai", ignoreCase = true)) {
    // MODE VRAI/FAUX : Une colonne de boutons larges
    question.reponses.forEach { reponse ->
        Button(
            onClick = { onReponseClick(reponse.id) },
            modifier = Modifier
                .fillMaxWidth( fraction = 0.8f) // Pas trop large non plus
                .padding(vertical = 8.dp)
                .height(60.dp)
        ) {
            Text(text = reponse.libelle, fontSize = 18.sp)
        }
    }
} else {
    // MODE QCM : Une grille de 2x2
    question.reponses.chunked( size = 2).forEach { paire ->
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            paire.forEach { reponse ->
                Button(
                    onClick = { onReponseClick(reponse.id) },
                    modifier = Modifier
                        .weight(1f) // 50% de la largeur chacun
                        .padding(vertical = 6.dp)
                        .height(80.dp)
                ) {
                    Text(text = reponse.libelle, textAlign = TextAlign.Center)
                }
            }
        }
```

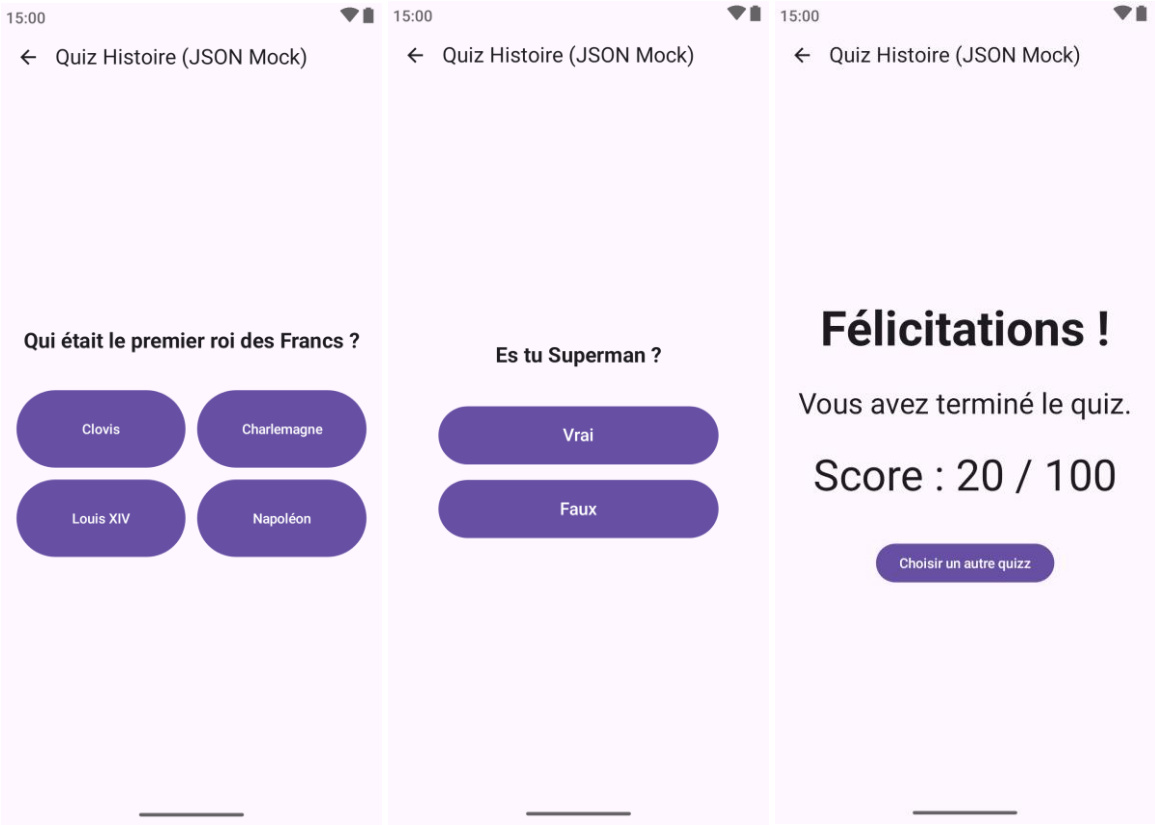
```
            }
        }
    } else {
        // ÉCRAN DE FIN DE QUIZ
        Text("Félicitations !", fontSize = 50.sp, fontWeight = FontWeight.Bold)
        Spacer(modifier = Modifier.height(30.dp))
        Text("Vous avez terminé le quiz.", fontSize = 30.sp)
        Spacer(modifier = Modifier.height(30.dp))
        Text("Score : $score / $scoreTotal", fontSize = 45.sp)
        Spacer(modifier = Modifier.height(40.dp))
        Button(onClick = onBackClick) { Text("Choisir un autre quizz") }
    }
}
}
```

Les preview :

```
// --- 3. LA PREVIEW (Simulée avec de fausses données) ---
@Preview(showBackground = true, showSystemUi = true, name = "Preview QCM")
@Composable
fun PreviewJeu() {
    // On simule l'état où le quiz est chargé et on affiche la question 0
    val quiz = MockData.questionnaireTest
    val premiereQuestion = quiz.questions[0]

    ContenuJeu(
        quizLibelle = quiz.libelle,
        question = premiereQuestion, // 👈 On passe la question au lieu du quiz entier
        isLoading = false,
        score = 0,
        scoreTotal = 100,
        onBackClick = {},
        onReponseClick = {}
    )
}

@Preview(showBackground = true, showSystemUi = true, name = "Preview Vrai/Faux")
@Composable
fun PreviewJeuVF() {
    // Utile pour tester si tes 2 boutons s'affichent bien différemment des 4 boutons
    val quiz = MockData.questionnaireTest
    val secondeQuestion = quiz.questions[1]

    ContenuJeu(
        quizLibelle = quiz.libelle,
        question = secondeQuestion,
        isLoading = false,
        score = 0,
        scoreTotal = 100,
        onBackClick = {},
        onReponseClick = {}
    )
}
```

```
@Preview(showBackground = true, showSystemUi = true, name = "Preview Fin jeu")
@Composable
fun PreviewFinJeu() {
    // Utile pour tester si tes 2 boutons s'affichent bien différemment des 4 boutons
    val quiz = MockData.questionnaireTest


    ContenuJeu(
        quizLibelle = quiz.libelle,
        question = null,
        isLoading = false,
        score = 20,
        scoreTotal = 100,
        onBackClick = {},
        onReponseClick = {}
    )
}
```

```
object MockData {
    val questionnaireTest = QuizzDetail(
        id = 1,
        libelle = "Quiz Histoire (JSON Mock)",
        questions = listOf(
            Question(
                id = 101,
                libelle = "Qui était le premier roi des Francs ?",
                type = TypeQuestion( id = 1, libelle = "QCM"),
                reponses = listOf(
                    Reponse( id = 1,  libelle = "Clovis",  estCorrecte = true),
                    Reponse( id = 2,  libelle = "Charlemagne",  estCorrecte = false),
                    Reponse( id = 3,  libelle = "Louis XIV",  estCorrecte = false),
                    Reponse( id = 4,  libelle = "Napoléon",  estCorrecte = false)
                ),
                niveaux = Niveau( id = 1,  nom = "Debutant",  nbpoints = 1),
                themes = listOf(Theme( id = 1,  libelle = "Cybersecurité")),
            ),
            Question(
                id = 102,
                libelle = "Es tu Superman ?",
                type = TypeQuestion( id = 2, libelle = "Vrai/Faux"),
                reponses = listOf(
                    Reponse( id = 1,  libelle = "Vrai",  estCorrecte = true),
                    Reponse( id = 2,  libelle = "Faux",  estCorrecte = false)
                ),
                niveaux = Niveau( id = 1,  nom = "Debutant",  nbpoints = 1),
                themes = listOf(Theme( id = 1,  libelle = "Cybersecurité")),
            )
        )
    )
}
```



Les résultats du questionnaires

Cependant, dans la version finale il faudrait pouvoir récupérer les données issues du site fait en Cake PHP.

API JSON – Cake PHP & Android

Avec les fonctions json de la classe QuestionnairesController, nous allons pouvoir récupérer les données du questionnaire, des questions, du niveau (les points gagnés par question), du type de question ainsi que leur thème, le nom et leur date de création pour pouvoir les insérer dans notre application Android.

```
{
    "questionnaires": [
        {
            "id": 5,
            "libelle": "Questionnaire3",
            "date_creation": "08\/04\/2026",
            "nb_questions": 1
        },
        {
            "id": 6,
            "libelle": "Questionnaire4",
            "date_creation": "08\/04\/2026",
            "nb_questions": 1
        },
        {
            "id": 2,
            "libelle": "questionnaire pas important",
            "date_creation": "01\/04\/2026",
            "nb_questions": 3
        },
        {
            "id": 7,
            "libelle": "Questionnaire5",
            "date_creation": "08\/04\/2026",
            "nb_questions": 1
        },
        {
            "id": 1,
            "libelle": "Questionnaire de r\u00e9ussite",
            "date_creation": "31\/03\/2026",
            "nb_questions": 4
        }
    ]
}
```


Le fichier JSON qui permet de récupérer la liste des questionnaires et l'afficher comme sur l'écran 2.


```
{
    "questionnaire": {
        "id": 1,
        "libelle": "Questionnaire de r\u00e9ussite",
        "questions": [
            {
                "id": 4,
                "libelle": "Va-t-on r\u00e9ussir le BTS ???",
                "type_id": 1,
                "niveau_id": 6,
                "_joinData": {
                    "id": 5,
                    "questionnaire_id": 1,
                    "question_id": 4,
                    "created": "2026-04-08T04:25:45+00:00",
                    "modified": "2026-04-08T04:25:45+00:00"
                },
                "type": {
                    "id": 1,
                    "libelle": "QCM"
                },
                "reponses": [
                    {
                        "id": 5,
                        "libelle": "Un jour maybe",
                        "iscorrect": true,
                        "question_id": 4
                    },
                    {
                        "id": 6,
                        "libelle": "Noooooon ",
                        "iscorrect": false,
                        "question_id": 4
                    }
                ],
                "themes": [],
                "niveaux": {
                    "id": 6,
                    "niv": "Difficile ",
                    "nbpoints": 3
                }
            },
```


viewJson permet de récupérer toutes les questions d'un questionnaire choisit ainsi que les informations complémentaires comme son id, son libellé, si c'est la bonne ou mauvaise réponse ou si c'est du type QCM ou Vrai/Faux.

```
// Le "contrat" de notre API. Définit les routes (URL) et ce qu'elles renvoient.
2 Usages
interface QuizzApiService {

    // Route : /questionnaires/index-json.json
    1 Usage
    @GET( value = "questionnaires/index-json.json")
    suspend fun getListeQuiz(): ApiReponseListeQuestionnaires

    // Route : /questionnaires/view-json/{id}.json
    1 Usage
    @GET( value = "questionnaires/view-json/{id}.json")
    suspend fun getQuizDetail(@Path( value = "id") id: Int): ApiReponseDetailQuestionnaire
}
```

```
3 Usages
object RetrofitClient {
    1 Usage
    private const val BASE_URL = "http://10.0.12.15/quizz/"

    // lazy = l'instance ne sera créée que la première fois qu'on l'appelle
    val instance: QuizzApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create()) // Transforme le JSON en Objets Kotl
            .build()
            .create(QuizzApiService::class.java)
    }
}
```

RetrofitClient permet au système de récupérer les informations du fichier JSON depuis le lien de ma machine virtuelle où j'ai fait le site en CakePHP. Le pare-feu doit avoir une modification pour pouvoir accéder à la ressource en dehors de la VM.

Ainsi, lorsque l'application sera lancée, les données de l'application seront mis à jour en fonction des données mis en ligne sur le site CakePHP.